Experiment: CI/CD workflow report

Course outcome: CO1

This report describes the complete continuous integration / continuous delivery workflow built in Lab 1: pipeline architecture, stages, automation results and observations, with supporting screenshots.

1. Objective

Build a foundational DevOps workflow for a sample Flask application: Git-based source control on GitHub, a Jenkins CI/CD pipeline that builds, tests, packages and deploys the application automatically, and a clean, reproducible setup documented end to end.

2. Tools and environment

- Ubuntu 24.04 LTS, Git 2.43.0, Python 3.12 + Flask 3.1, pytest
- Jenkins 2.580.2 LTS on Java 21 (local server, port 8080)
- GitHub repository: github.com/theabhinav0231/devops-lab-1
- systemd for deployment (devops-lab1.service on port 5000)

3. Pipeline architecture

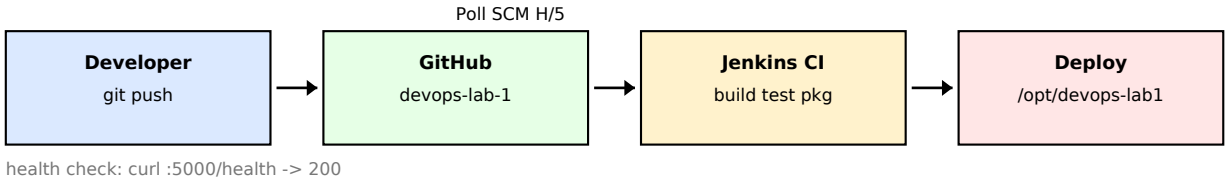


Fig: CI/CD workflow architecture of the lab

The developer pushes commits to GitHub. Jenkins polls the repository every 5 minutes (H/5 * * * *); when a change is detected the pipeline checks out the code, builds it, runs the tests, packages an artifact and deploys it to the local systemd service, finishing with a health check against http://localhost:5000/health.

4. Pipeline stages

#	Stage	Purpose	Result
1	Checkout	Pull latest code from GitHub (checkout scm)	OK
2	Build	Create virtualenv, install Flask	OK
3	Test	Run pytest unit tests (2 tests)	OK

4	Package	Create app-build-N.tar.gz, archive artifact	OK
5	Deploy	Copy to /opt/devops-lab1, restart service, health check	OK

5. Automation results

Build	Trigger	Commit	Result	Duration
#1	Build Now	8946b3f (Jenkinsfile + deploy script)	SUCCESS	29 s
#2	Poll SCM (auto)	7530c85 (bump app version to 1.0.1)	SUCCESS	17 s

After build #2 the running application reported version 1.0.1, proving the commit went from git push to a running service without any manual step.

```
$ curl http://localhost:5000/health
{"status":"UP","version":"1.0.1"}
```

6. Evidence

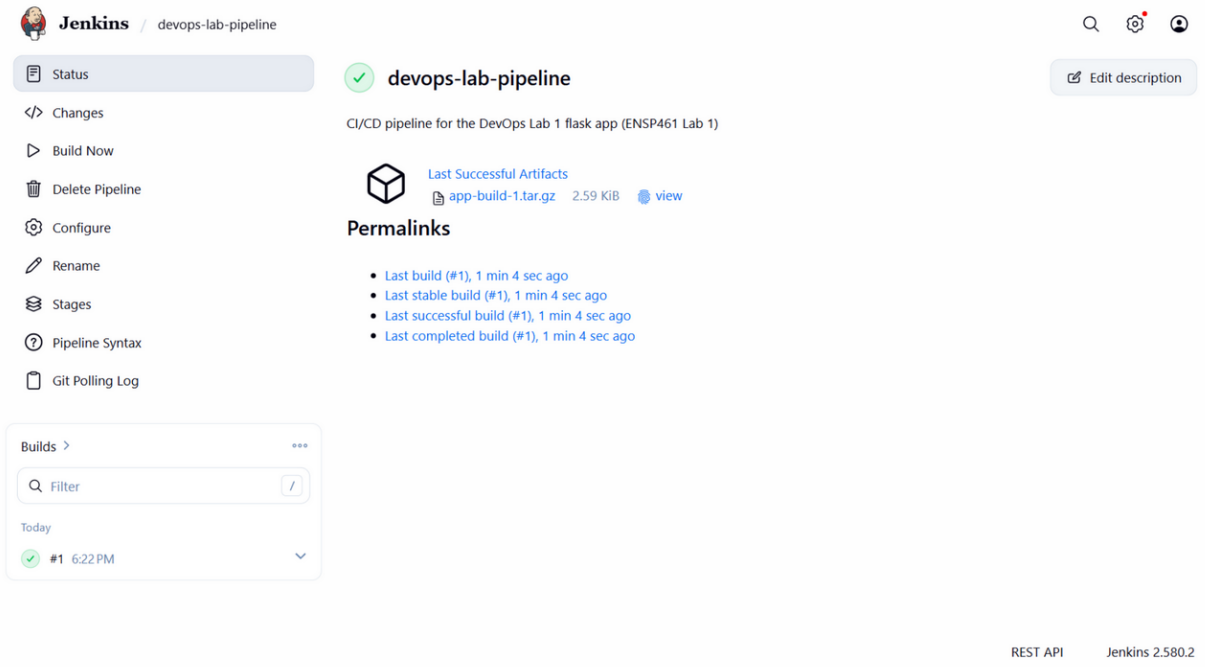


Fig: Jenkins pipeline job with build #1

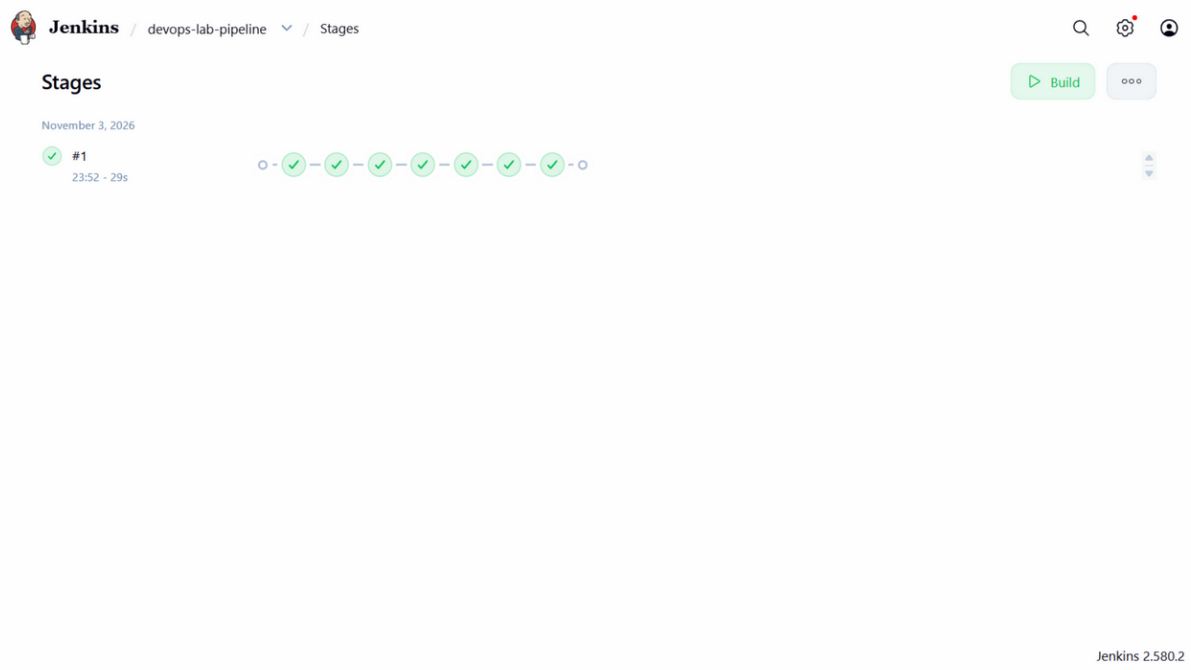


Fig: Stage graph - every stage green

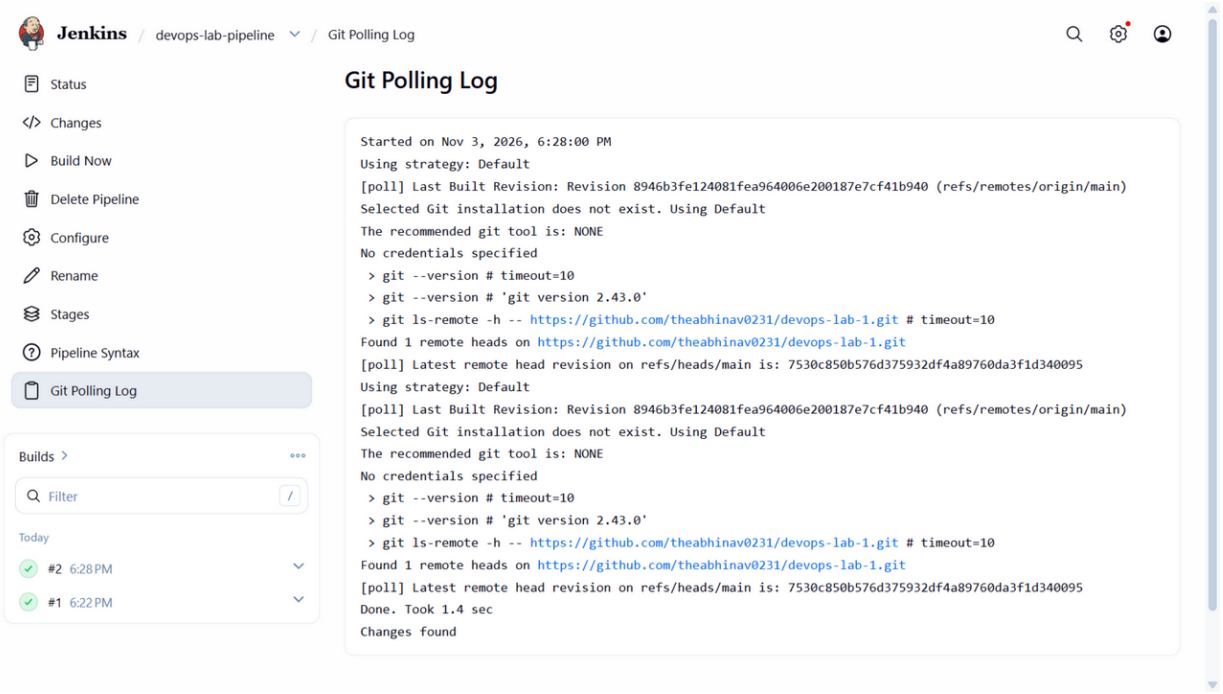


Fig: Polling log proving the automatic trigger

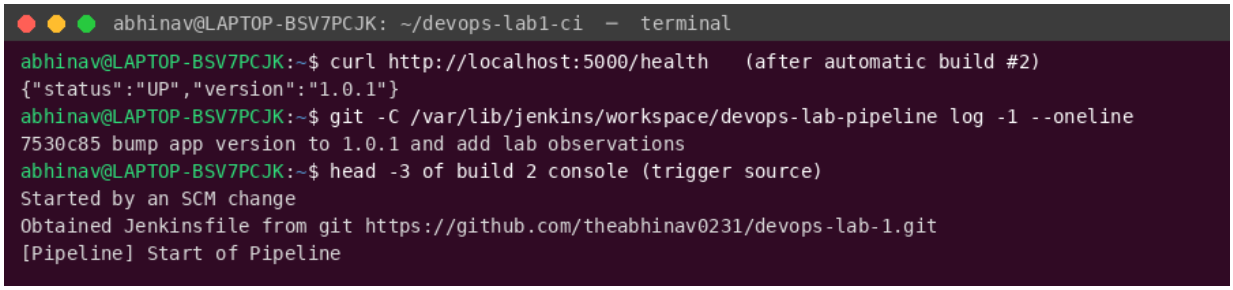


Fig: Health check after automatic deployment

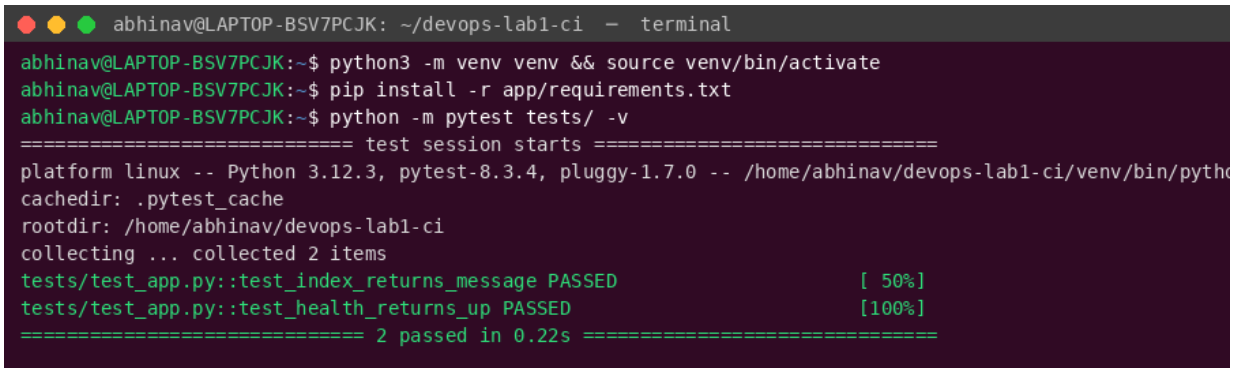


Fig: Unit tests passing locally (same suite runs in the pipeline)

7. Observations

- Keeping the Jenkinsfile in the repository (Pipeline script from SCM) makes the pipeline itself version controlled and reproducible.
- Poll SCM (H/5) was used instead of a webhook because the Jenkins server runs locally and is not reachable from GitHub.
- The health check inside the deploy script converts a silent bad deployment into a visible pipeline failure.
- Meaningful commit messages (initial commit, add pytest unit tests, add Jenkinsfile..., bump app version to 1.0.1) make the history readable.
- Total cycle time from push to running new version was under 6 minutes (poll interval + ~17 s build).

8. Conclusion

A working CI/CD pipeline was implemented: source control on GitHub, automated build/test on every detected change, artifact packaging, and automated deployment verified by a health check. All evidence (commands, logs, screenshots, commit history) is available in the repository under docs/.